Date _____ 20____
M T W T F S S

Q1 (a) 2. for 1 element no comparisions

$C(1) = 0$

for 2 elements 1 comparision

$C(2) = 1$

Recurrence relation:

$C(n) = 2C(n/2) + 2$

$C(n) = 2[2C(n/4) + 2] + 2$

$= 4C(n/4) + 4 + 6$

$C(n) = 2[4C(n/8) + 2] + 2$

$= 8C(n/8) + 14$

$C(n) = 2^i C(n/2^i) + 2(2^i - 1)$

$n = 2^4$

$n/2^i = 2$

$2^i = n/2$

$C(n) = n/2\, C(2) + 2(n/2 - 1)$

Since $C(2) = 1$

$C(n) = n/2 + 2(n/2 - 1)$

$= n/2 + n - 2$

$= 3n/2 - 2$

TICK INDUSTRIES PRIVATE LIMITED

CamScanner

Date _____ 20 ____
M T W T F S S

Take n = 8

$$C(8) = \frac{3(8)}{2} - 2 = 10$$

Time Complexity : $C(n) = O(n)$.

21a) 3. The divide and conqure algorithm is more comparision efficient than brute force because it processes elements in pairs and combines the results using only 2 comparisions per recursive combinations. Both algo have $O(n)$ time complexity but divide and conqure performs fewer key comparisions.

Date _____ 20 ____
M T W T F S S

Q1 b) 2. When n = 1, algo returns a only

$M(0) = 0$

$M(n) = M(n/2) + 1$ → for even n
2 → for odd n

$M(n) = M(n/2) + 1$

$M(n) = M(n/4) + 2$

$M(n) = M(n/8) + 3$

$M(n) = M(n/2^k) + k$

Since $n = 2^k$

$m(n) = M(1) + k$
$M(n) = k$

$k = \log_2 n$

$M(n) = \log_2 n$

$M(n) = O(\log n)$

TICK® INDUSTRIES PRIVATE LIMITED

CamScanner

Date _____ 20 ____
M T W T F S S

Q1b) 3. The algorithm used is more efficient than brute force because it reduces the exponent approximately by half at each recursive call. It multiplication count grows logarithmically, whereas the brute force requires a linear number of multiplications. So the used divide and conquer is better for large exponents.

CamScanner

Date _____ 20 ____
M T W T F S S

Q1d) Best case

It occurs when the pivot divides the array into two equal parts at every recursive step.

$$T(n) = 2T(n/2) + O(n)$$

Using Masters theorem:

$$T(n) = aT(n/b) + f(n)$$

$$a = 2,\ b = 2,\ f(n) = \Theta(n)$$

$$n^{\log_b a} = n$$

$$f(n) = \Theta(n^{\log_b a})$$

$$T(n) = \Theta(n^{\log_b a} \log n)$$

$$T(n) = \Theta(n^1 \log n)$$

$$T(n) = \Theta(n \log n)$$

Worst case

Worst case is when a recursive call splits the array into uneven parts

$$T(n) = T(n-1) + O(n)$$
$$T(n) = O(n) + O(n-1) + \ldots + O(1)$$

$$T(n) = O\left(\frac{n(n+1)}{2}\right)$$

$$T(n) = O(n^2)$$

TICK INDUSTRIES PRIVATE LIMITED

CamScanner

Date _____ 20 ____
M T W T F S S

Q1 c) 2.) No, it is not the best choice for 1D ~~&~~ CPP as the numbers can be sorted and the closest pair must consist of adjacent elements

Date _____ 20 ____
M T W T F S S

Q1 f) Problem understanding:

A unimodal array is an array whose value first increase then decrease. The max element is called peak.

An example is:

$$A = [2, 5, 8, 12, 9, 6, 3]$$

The objective is to find the peak by reading at most $O(\log n)$ array entries.

Solution:

This problem can easily be solved by divide and conquer. Instead of checking every element, we examine the middle element and its 2 neighbours. These values tell us which half of the array contain peak.

Case 1: Increasing slope, If the middle element is greater than its left neighbor and smaller than its right, then that mean peak is on the right, discard left side.

Case 2: Decreasing slope, If the middle element is smaller than its left and greater than its right neighbour then discard the right side.

Case 3: Peak found, If the middle element is greater than both neighbours then it is the peak.

TICK

CamScanner

Date _____ 20 ____
M T W T F S S

Dry run

Take array:

$A = [2, 5, 8, 12, 9, 6, 3]$

Iteration 1:

- middle index = $7/2 = 3$
- $A[3] = 8$
- Its neighbours are 5 and 12
- Since increasing slope discard left side.

Iteration 2:

12 9 6 3

- middle index = $(4+7)/2 = 5$
- $A[5] = 9$
- Its neighbours are 12 and 6
- Decreasing slope so discard right side.

Iteration 3:

- Remaining element $A[4] = 12$
- It's the peak.

peak = $\boxed{12}$

Date _____ 20 ____
M T W T F S S

Time Complexity:

$$T(n) = T(n/2) + c$$

$$T(n) = T(n/2) + c$$
$$= T(n/4) + 2c$$
$$= T(n/8) + 3c$$

$$n/2^k = 1$$
$$2^k = 1$$
$$k = \log_2 n$$

$$T(n) = O(\log n)$$

Pseudocode:

```
Algorithm FindPeak(A, low, high)
    if low = high
        return low
mid = floor((low+high)/2)

if A[mid] > A[mid-1] and A[mid] > A[mid+1]
    return mid

else if A[mid] < A[mid+1]
    return FindPeak(A, mid+1, high)

else
    return FindPeak(A, low, mid-1).
```

TICK INDUSTRIES PRIVATE LIMITED

CamScanner

Date _____ 20____
M T W T F S S

Q10)

- Stock prices are given for n days consecutive
- Need to find a day to best for buying one share.
- Need to find a later day to sell the share to maximize profit.
- Profit per share is:

$$\text{Profit} = p(j) - p(i), \quad i < j., \quad i = \text{buying day}, \quad j = \text{selling day}.$$

- Company buys 1000 shares so $\text{Profit} = 1000[p(j) - p(i)]$.
- Objective is to solve in $O(n \log n)$ time.

Solution:

- We divide the sequence of stock prices into two halfs and recursively find the best transaction in each half.

- However the best transaction can be buying in the first half and selling in the second

- So Consider 3 possibilities:
  1) Best transaction in first half
  2) " " second half
  3) " " across halves.

- for 3rd possibility, we find the min stock price in the left and the max stock price in right half.

Date _____ 20 ____
M T W T F S S

Pseudocode:

Algorithm MaxProfit(P, low, high):

If low = high
return ~~low~~ no transaction

mid = floor((low + high)/2)

leftProfit = MaxProfit(P, low, mid)
rightProfit = MaxProfit(P, mid+1, high)

minPrice : minimum price in P[low .... mid]
maxPrice : maximum price in P[mid+1 ..... high]

crossProfit = maxPrice - minPrice

return the best of leftProfit, RightProfit, crossProfit

Dry run:

P = [9, 1, 5]

Step 1

Divide the array

~~for~~ LH RH
[9, 1] [5]
Days 1-2 Day 3

TICK
INDUSTRIES PRIVATE LIMITED

CamScanner

Date _____ 20 ____
M T W T F S S

Step 2
Find the best transaction in each half
LH: Buy at 9$ on day 1 and sell at $1 on day 2, $8 loss.

RH: Same day so buy and sell on different days impossible.

Step 3
· find the cross half transaction
· The min price on LH is $1 on day 2
· The max price on RH is $5 on day 3

Cross profit = 5-1 = $4

Best transaction is Buy on day 2, sell on day 3.

Time Complexity:

$$T(n) = 2T(n/2) + O(n)$$

Using the Master theorem,

$$a = 2, b = 2, f(n) = O(n)$$

$$n^{\log_b a} = n^{\log_2 2} = n$$

Since $f(n) = O(n)$, Case 2 is satisfied

$$T(n) = O(n \log n)$$

b). $O(n \log n)$

Date _____ 20 ____
M T W T F S S

Q1 h) Problem 1:

- Use the binary search to find how many elements of the medians first n elements come from each database.

- If we select i elements from DB A then we must select n-i elements from DB B.

- Need to find the partition where all selected elements are smaller than all unselected ones.

Pseudocode:

```
Algorithm findMedian(A, B, n)
    low = 0
    high = n

    while low <= high
        i = floor((low+high)/2)
    j = n - i

    if i == 0
        leftA = -infinity
    else
        leftA = Query(A, i)

    if i == n
        rightA = +infinity
    else
        rightA = Query(A, i+1)
```

Date _____ 20 ____
M T W T F S S

```
if j == 0
    leftB = -infinity
else
    leftB = Query(B, j)

if j == n
    rightB = +infinity
else
    rightB = Query(B, j+1)

if leftA < rightB and leftB < rightA
    return mca(leftA, leftB)

else if leftA > rightB
    high = i - 1

else
    low = i + 1
```

Complexity

The number of iterations is O(log n)

Q). O(log n)

Date _____ 20____
M T W T F S S

Problem 3:

- Use divide and conquer
- Recursively find majority candidate in 1st half
- " " in 2nd half.
- Combine the results.
- If an overall majority candidate exists it must be among the candidates returned by the 2 recursive calls.
- If both halves return the same candidate that candidate has overall majority.

Pseudocode:

```
Algorithm MajorityCand(C, low, high)

    if low == high:
        return C[low]
    mid = floor((low+high)/2)

    leftCand = MajorityCand(C, low, mid)
    rightCand = "          "(C, mid+1, high)

    if leftCand and rightCand are equivalent
        return leftCand.

    leftCount = 0
    rightCount = 0

    for i = low to high

        if leftCand exists and C[i] is equivalent to leftCand
            leftCount = leftCount + 1
```

TICK INDUSTRIES PRIVATE LIMITED

CamScanner

Date _____ 20 ____
M T W T F S S

if rightCand exists and C[i] is equivalent to RightCand
    rightCount = rightCount + 1

if leftCount > (high - low + 1) / 2
    return leftCand

if RightCount > (high - low + 1) / 2
    return ~~RightCount~~ RightCand

return NO_Mgority.

Problem 2:

An ordinary pair inversion is a pair of indices $i < j$ for which

$$a_i > a_j$$

A Significant inversion is more restrictive. It is a pair satisfying

$$i < j \text{ and } a_i > 2a_j$$

A brute force algo will check every pair for $O(n^2)$. So we can use a merge sort variation which will use the divide and conquer technique. The solution divides the array in 2 halves and recursively counts significant inversions in each half.

3 sources of significant inversions:

1) Pair within the left half
2) Pair within the right half
3) Pair with one element in left and one in right.

We need an efficient method for counting the third category.

After recursively sorting both halves, we can count cross-half significant inversions using 2 pointers. Because both halves are now sorted, the pointer only moves forward. We merge the halves normally.

This allows to count inversions in linear time

Pseudocode:

```
Algorithm Count Significant(left, right, A)
    if left >= right
        return 0

    mid = floor((left + right) / 2)

    count = Count Significant(left, mid, A) + Count Significant(mid+1, right, A)

    j = mid + 1
    for i = left to mid
        while j <= right and A[i] > 2 * A[j]
            j = j + 1
        count = count + (j - (mid + 1))
```

Merge A[left.... mid] and A[mid+1.... right]

return count.

Dry Run

Consider

$A = [1, 3, 2, 3, 1]$

$LH = [1, 3, 2]$, $RH = [3, 1]$

After being sorted Both halves become

$LH = [1, 2, 3]$, $RH = [1, 3]$

left value n	check	New Inversions
1	$1 > 2(i)$? No	0
2	$2 > 2(1)$? No	0
3	$3 > 2(i)$? Yes	1

for $n = 3$, the right half value 1 qualifies, but 3 does not, because $3 > 2(3)$ is false.
Thus only 1 cross significat inversion: The original 3 at index 2 and the 1 at index 5.

- LH [1, 3, 2]: 0 Significat inversion
- RH [3, 1]: 1 Significat inversions. because $3 > 2(1)$

Total = 0 + 1 + 1 = 2.

Time Complexity:

$$T(n) = 2T(n/2) + O(n)$$

$a = 2, b = 2 \quad f(n) = \Theta(n)$

$$n^{\log_b a} = n^{\log_2 2} = n$$

Since $f(n) = \Theta(n)$ Case 2 applies.

$$T(n) = \Theta(n \log n)$$

Before merging we can count all the inversions in $O(n \log n)$ time instead of $O(n^2)$.

CamScanner